

Stage 10 — Golden Drawer System

Status: IMPLEMENTED / READY FOR EXTERNAL REVIEW

Stage boundary: Stop after Stage 10. Stage 11 was not started.

1. Resumo executivo

A Stage 10 adiciona uma fundação paramétrica declarativa para gavetas sem duplicar o engine de construção, sem criar uma segunda fonte de verdade e sem alterar a arquitetura Registry + Dispatch da Stage 9. O piloto escolhido é o `kitchen-drawer-3`, uma `ModuleDefinition` que já existia no Dioris. Não foi criado um novo módulo de gaveteiro.

O caminho final é:

Carcass Opening → **DrawerStackRule** → **DrawerSlideApplicationRule** → **Drawer Fronts**
+ **Drawer Boxes** + **Slides**

A geometria de gavetas é resolvida por regras e resolvers puros. A materialização continua usando `PartDefinition`, `FurnitureInstance`, o builder e os serviços downstream existentes. A identificação física é instance-scoped após o build; o dispatch usa somente `moduleDefinitionId`.

2. Auditoria do drawer existente

A auditoria encontrou `buildDrawers` em `builders.ts` e as definições existentes `kitchen-drawer-1`, `kitchen-drawer-2`, `kitchen-drawer-3` e `kitchen-drawer-4`. O builder legado já gerava fronts, laterais, traseiras, fundos, slides e puxadores, mas mantinha quantidade, altura, folgas, largura de caixa e profundidade em fórmulas inline.

A base reutilizada também inclui os roles `drawer-front`, `drawer-side` e `drawer-bottom`, metadados de interação `interactive.type: "drawer"`, `maxTravelMm`, `groupId`, `openStates`, `toggleInstanceAnimation`, `validateOpeningClearance`, joinery, machining, BOM, cut-list e nesting.

Capability	Resultado da auditoria	Decisão
ModuleDefinition	<code>kitchen-drawer-3</code> já existe	Reutilizar
Drawer builder	Existente, porém inline/legacy	Encaminhar o piloto para rules/resolver
Carcass	Resolver compartilhado existente	Reutilizar
Motion	<code>FurnitureInstance</code> + opening validator existentes	Reutilizar
BOM/cut-list/nesting	Consumidores reais já existentes	Reutilizar e testar
Slides industriais	IDs genéricos existem, sem manufacturing variants verificadas	Não inventar especificação

3. Golden Drawer Pilot e ModuleDefinition

O piloto canônico é `kitchen-drawer-3`, com três gavetas. O identificador não foi duplicado e a definição profissional original permanece no catálogo Kitchen. A regra de quantidade é declarativa no profile; a persistência continua armazenando a instância e o `moduleDefinitionId`, não um resultado resolvido.

O profile criado é `kitchen-drawer-3:construction-profile-v1`. Ele referencia a `GOLDEN_DRAWER_CARCASS_CONSTRUCTION_RULE`, `GOLDEN_DRAWER_3_STACK_RULE`, `GOLDEN_DRAWER_3_BOX_RULE` e `GOLDEN_DRAWER_3_SLIDE_RULE`.

4. ConstructionProfile e Registry

`ConstructionProfile` foi estendido com três slots opcionais:

Slot	Função
<code>drawerStackRule</code>	Quantidade, distribuição, reveals e gaps
<code>drawerBoxRule</code>	Espessuras e reduções da caixa
<code>drawerSlideApplicationRule</code>	Clearance, profundidade, travel policy e readiness industrial

O Registry valida ownership de todos os novos rules contra `moduleDefinitionId`. Não guarda `ResolvedDrawerStack`, peças materializadas ou dimensões de uma instância. Um profile incompleto ou ausente para uma definição profissional resulta em hard stop; não é permitido cair silenciosamente no builder legacy.

5. Carcass e opening

O drawer usa a mesma engine `resolveCarcassConstruction`. A regra de carcass do piloto usa `full-height-above-toe-kick` e `separate-profile-supported-by-feet`, portanto conserva o comportamento de gaveteiro inferior com rodapé real.

A abertura é obtida de `ResolvedCarcass`:

```
internalWidth  = externalWidth - 2 × panelMm
bodyHeight     = externalHeight - toeKickMm
internalHeight = bodyHeight - 2 × panelMm
internalDepth  = externalDepth - backMm
```

Nenhuma segunda fórmula de carcass foi criada no Drawer resolver.

6. DrawerStackRule e ResolvedDrawerStack

A regra do piloto é `equal`, com três gavetas, reveal superior de 2 mm, reveal inferior de 2 mm e gap entre frentes de 2 mm. Esses valores são uma regra paramétrica do piloto visual existente; não são dados de fabricante.

Para `N` gavetas, a equação usada é:

```
frontHeight =
    (internalHeight - topReveal - bottomReveal - (N - 1) × interDrawerGap) ,
```

Cada item resolvido possui `drawerId`, `frontId`, bounds verticais, centro, largura de frente, largura/altura/profundidade de caixa, clearances de slide e travel visual seguro. Quantidade, largura e altura variam sem hardcode por tamanho externo.

Os testes cobrem `800 → 900 → 800`, variações de largura `600/800/900/1000`, alturas `720/870/900`, quatro gavetas em resolver isolado, gaps negativos, contagem zero, abertura pequena e profundidade insuficiente.

7. Drawer fronts

Cada gaveta possui uma frente decorativa `drawer-front` separada da caixa. As três frentes recebem IDs estáveis `drawer-1:front`, `drawer-2:front` e `drawer-3:front` no escopo de definição; o pós-build converte os IDs para o `instanceId` da ocorrência.

As frentes pertencem ao mesmo `FurnitureInstance` que a carcass, a caixa e as ferragens. Possuem `groupId`, `interactive.type: "drawer"` e `maxTravelMm`.

8. Drawer boxes

Cada gaveta possui uma frente estrutural própria `drawer-box-front`, duas laterais, uma traseira e um fundo. A frente decorativa não é usada como frente estrutural da caixa.

O novo role `drawer-box-front` foi adicionado ao contrato de `PartDefinition`, recebeu grain vertical e espessura de painel no normalizador downstream, e já aparece corretamente no cut-list e nesting.

A largura da caixa é derivada por:

```
boxWidth = internalWidth - lateralClearanceLeft - lateralClearanceRight
```

A altura da caixa é derivada por:

```
boxHeight = frontHeight - sideHeightReduction
```

A profundidade visual preserva a regra existente:

```
boxDepth = internalDepth - depthClearance
```

9. Slide abstraction e honestidade industrial

O Registry possui os IDs semânticos `slide-telescopic`, `slide-hidden` e `slide-hidden-soft-close`. O piloto usa `slide-hidden-soft-close` como hardware visual genérico. A regra declara 13 mm de clearance lateral por lado e 60 mm de recuo de profundidade para preservar o resultado visual preexistente do builder.

Esses valores não são apresentados como especificação de fabricante. O

`HardwareRegistry` atual não possui manufacturer/model/code/technical data verificável para essas correções. Consequentemente:

Resultado	Estado
Geometria visual e placement semântico	READY
Motion visual e travel seguro	READY
BOM do ID genérico	READY
Manufacturing de correção	INCOMPLETE
Furação/machining industrial de correção	INCOMPLETE
Seleção de produto industrial	NÃO DECLARADA

A Stage 10 não escolhe Blum, Hettich, Häfele ou qualquer outra correção industrial sem revisão externa.

10. Motion, collision e interlock

A implementação reutiliza `interactive.type: "drawer"`, `groupId`, `maxTravelMm`, `openStates`, `toggleInstanceAnimation` e `validateOpeningClearance`. As seis peças móveis por gaveta — frente, frente estrutural, duas laterais, traseira e fundo — compartilham o mesmo grupo.

A suíte comprova três grupos independentes, travel positivo em todas as peças e atualização de `openStates` por grupo. A colisão de abertura continua delegada ao serviço existente; nenhum novo animation engine ou collision engine foi criado.

11. BOM, cut-list, nesting, joinery e machining

O acceptance test downstream utiliza os serviços reais:

Downstream	Resultado
BOM	6 slides e 3 puxadores no piloto
Cut-list	Frentes decorativas, <code>drawer-box-front</code> , laterais, traseiras e fundos com material, espessura e grain
Nesting	Integridade sem missing, duplicate ou unknown IDs; hardware não entra como placement de chapa
Joinery	Operações vinculadas à instância e às peças reais
Machining	Operações locais; readiness incompleta exige <code>missingParameters</code>
$A \rightarrow B \rightarrow A$	800 $\rightarrow$ 900 $\rightarrow$ 800 determinístico

12. Identidade e multi-instance

A seleção do profile ocorre por `moduleDefinitionId`. O `instanceId` só é usado na materialização e na reconstrução da ocorrência. Duas instâncias `kitchen-drawer-3` possuem IDs de peças disjuntos, operações locais equivalentes e permanecem isoladas após move/rotation.

A persistência/reload armazenou `moduleDefinitionId` e reconstruiu as 3 gavetas via Registry. Os IDs de `DrawerStackRule` e `DrawerBoxRule` não foram serializados na instância.

13. Regressões

A Stage 9 Registry regression permanece verde. Os módulos legados `kitchen-drawer-1`, `kitchen-drawer-2`, `kitchen-drawer-4` e `kitchen-base-door-drawer` continuam usando o caminho legado por não possuírem Drawer profile; o piloto `kitchen-drawer-3` usa o dispatch declarativo.

A regra profissional de drawer incompleta ou removida não cai no legacy: os locks de hard stop falham o build de forma rastreável.

14. Mutations

Foram executadas quatro mutations controladas, todas com restauração automática:

Mutation	Resultado
Remover <code>drawerStackRule</code> do profile	EXPECTED_FAIL
Trocar Definition ID por Instance ID no resolver	EXPECTED_FAIL
Colapsar <code>drawer-box-front</code> em <code>drawer-front</code>	EXPECTED_FAIL
Alterar a contagem declarativa de 3 para 2	EXPECTED_FAIL

Resumo persistido em `evidence/stage10-golden-drawer/summary.txt`.

15. Evidência técnica

`STAGE_10_GOLDEN_DRAWER_3_TECHNICAL.png` é uma **TECHNICAL DETERMINISTIC EVIDENCE**, gerada por desenho determinístico. Não é screenshot do Planner e não é apresentada como screenshot real. Ela mostra carcass, toe-kick, abertura interna, três frentes, gaps, uma caixa e a colocação semântica das correções.

16. Open-source provenance

Nenhum pacote ou implementação externa de gaveta foi copiado para o projeto. A base reutilizada é a arquitetura interna já existente no Dioris: `PartDefinition`, `ConstructionProfileRegistry`, `resolveCarcassConstruction`, `buildModule`, `validateOpeningClearance`, `joineryReport`, `machiningReport`, `fabricationReport` e `nestingPlan`.

Apenas os dados de fabricante já presentes no repositório para dobradiças/placas continuam com suas fontes oficiais existentes. As correções da Stage 10 permanecem genéricas e sem alegação industrial.

17. TypeScript, Vitest e build

A validação final executou `tsc --noEmit`, Vitest completo, production build e `git diff --check`.

Check	Resultado
Vitest	57 arquivos, 619 testes aprovados
Baseline anterior	606 testes
Testes adicionados nesta Stage	13
Production build	exit code 0
<code>git diff --check</code>	exit code 0
TypeScript	5 erros preexistentes, todos em <code>usePlannerStore.ts</code> linhas 1089–1093
Erros novos da Stage 10	0

Os cinco erros preexistentes são `TS2322` de `string | null` para `string` em `usePlannerStore.ts` ; nenhum aponta para os arquivos da Stage 10.

18. Arquivos principais

Arquivo	Papel
<code>contracts/DrawerRules.ts</code>	Contratos e resultados resolvidos
<code>contracts/ConstructionProfile.ts</code>	Slots declarativos de drawer
<code>families/kitchen/drawerRules.ts</code>	Rules Golden Drawer
<code>families/kitchen/carcassConstructionRules.ts</code>	Carcass rule do piloto
<code>services/drawerStackResolver.ts</code>	Opening e stack resolver puros
<code>families/kitchen/builders.ts</code>	Dispatch declarativo e materialização
<code>registry/ConstructionProfileRegistry.ts</code>	Ownership e hard stops
<code>pkg/state/stage10GoldenDrawerFoundation.test.ts</code>	Equações, invariantes, motion e identidade
<code>pkg/state/stage10GoldenDrawerAcceptance.test.ts</code>	Downstream e persistência
<code>scripts/stage10_mutation_checks.sh</code>	Quatro mutations controladas
<code>evidence/stage10-golden-drawer/STAGE_10_GOLDEN_DRAWER_3_TECHNICAL.png</code>	Evidência técnica determinística

19. Critério de encerramento

A Stage 10 está pronta para revisão externa porque fronts, boxes, hardware visual, BOM, cut-list e nesting derivam da mesma `FurnitureInstance`, por meio da abertura da carcass, das rules e do resolver declarativo. A industrialização definitiva das correções permanece explicitamente fora de READY até revisão e fonte de fabricante.

A Stage 11 não foi iniciada.